AIAC-14.4

Name: Siri Vempati

Ht.no:2303A51309

Batch.no:05

Task 1: Responsive Homepage Layout for a

Startup Website

Scenario

You are designing a basic homepage for a startup company that needs

to look good on desktops, tablets, and mobile devices.

Task Description

• Create a web page structure using HTML that includes:

o A header with navigation links

o A main content section

o A footer with basic information

• Use AI assistance to generate responsive CSS using media

queries.

• Ensure the layout adjusts smoothly across different screen sizes.

Expected Output

• A responsive web page where:

o Header, content, and footer are clearly separated

o Layout adapts correctly to mobile, tablet, and desktop

views

o Code is clean and well-structured

Prompt:

Create a web page structure using HTML that includes:

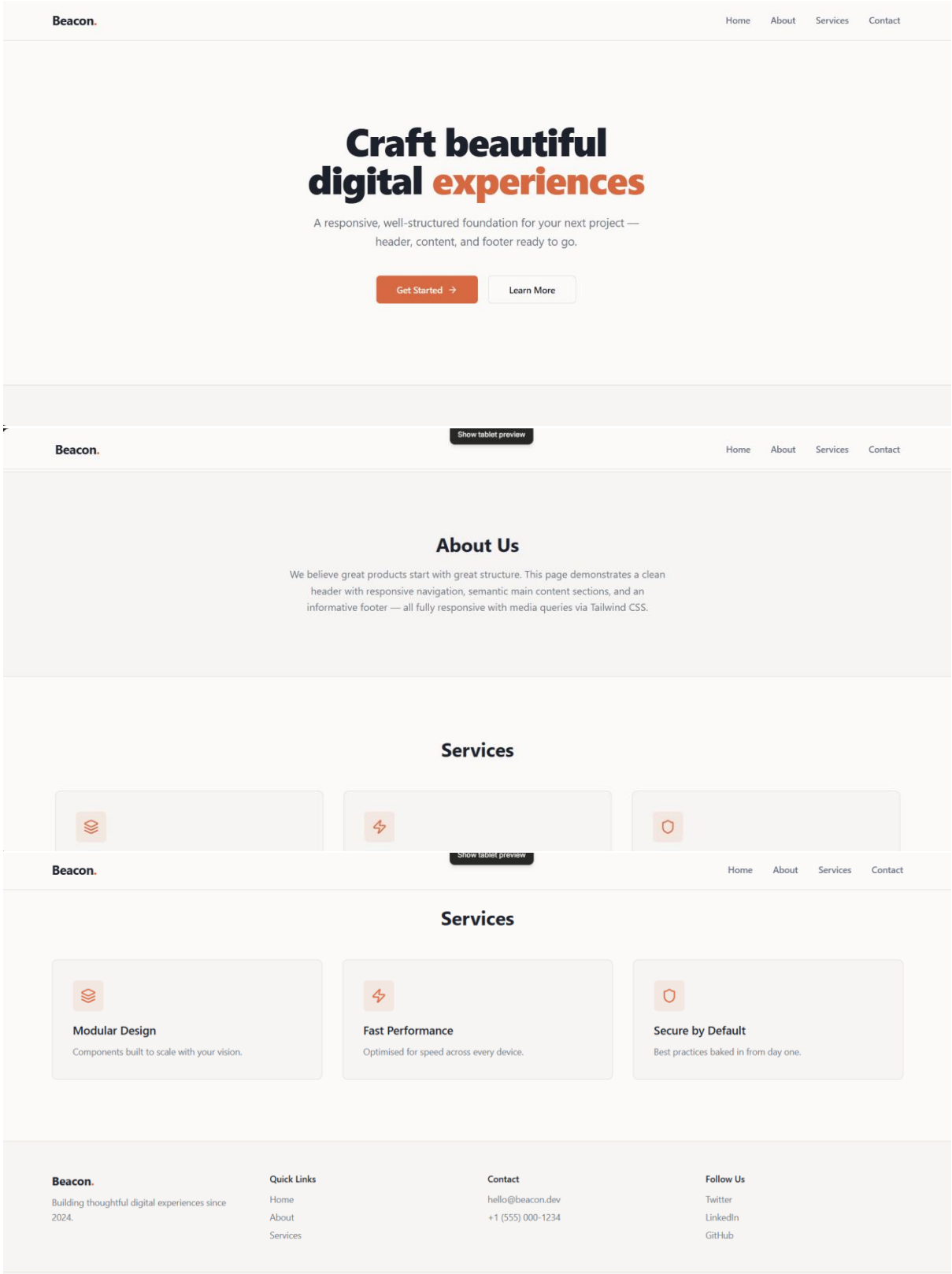
o A header with navigation links

o A main content section

o A footer with basic information

• Use AI assistance to generate responsive CSS using media queries

Output:



Explanation: This project creates a responsive homepage for a startup website using HTML and CSS.

- The HTML structure contains three main sections: header, main content, and footer.
- The header includes navigation links like Home, About, Services, and Contact.
- The main section displays the startup's welcome message.
- The footer contains copyright information.

The CSS uses Flexbox and media queries to make the layout responsive:

- Desktop: Navigation links appear horizontally.
- Tablet: Header and navigation stack vertically.
- Mobile: Text size and spacing adjust for smaller screens.

Task 2: Interactive Call-to-Action Button

Scenario

A marketing page requires a call-to-action button that responds when users interact with it.

Task Description

• Add a button to a web page using HTML.

• Use AI to generate JavaScript that responds to the button click by:

o Displaying a message or alert

• Ask AI to add meaningful comments explaining the JavaScript logic.

Expected Output

• A web page where:

o Clicking the button triggers a visible response (alert or message)

o JavaScript code is readable and commented

o No page reload occurs

Prompt:

Generate a simple web page with:

- A call-to-action button labeled "Get Started".

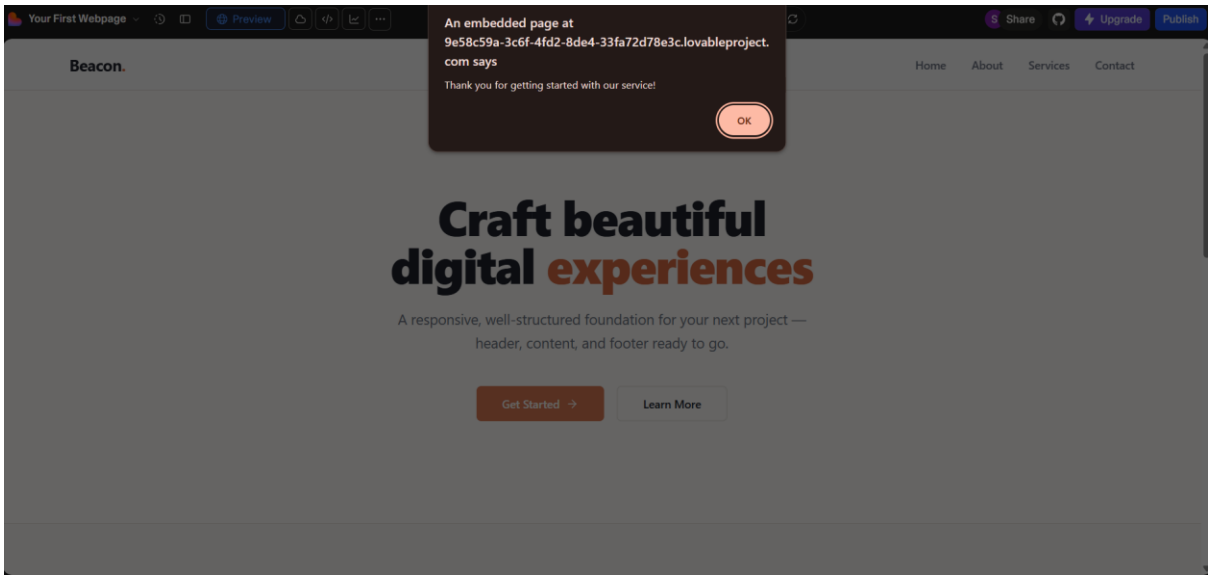
Use JavaScript so that when the user clicks the button:

- A message or alert appears saying "Thank you for getting started with our service!"

Ensure:

- The JavaScript includes meaningful comments explaining the logic.
- The button interaction does not reload the page.
- Code should be simple, clean, and beginner friendly.

Output:



Explanation:

This task creates an interactive call-to-action button using HTML and JavaScript.

- The HTML adds a button labeled "Get Started".
- JavaScript selects the button using getElementById().
- An event listener is added to detect when the button is clicked.
- When clicked, the browser displays an alert message thanking the user.
- The interaction happens without reloading the page.

Task 3: Contact Form with Client-Side

Validation

Scenario

You are developing a contact form for a company website to collect

user queries safely and correctly.

Task Description

• Design a form with fields for:

o Name

o Email

o Message

• Use AI to generate JavaScript validation logic that:

o Prevents empty submissions

o Checks for a valid email format

• Display clear error messages for invalid input.

Expected Output

• A functional form where:

o Invalid input shows inline error messages

o Valid input displays a success message

o Validation happens on the client side using JavaScript

Prompt:

Create a contact form using HTML with the following fields:

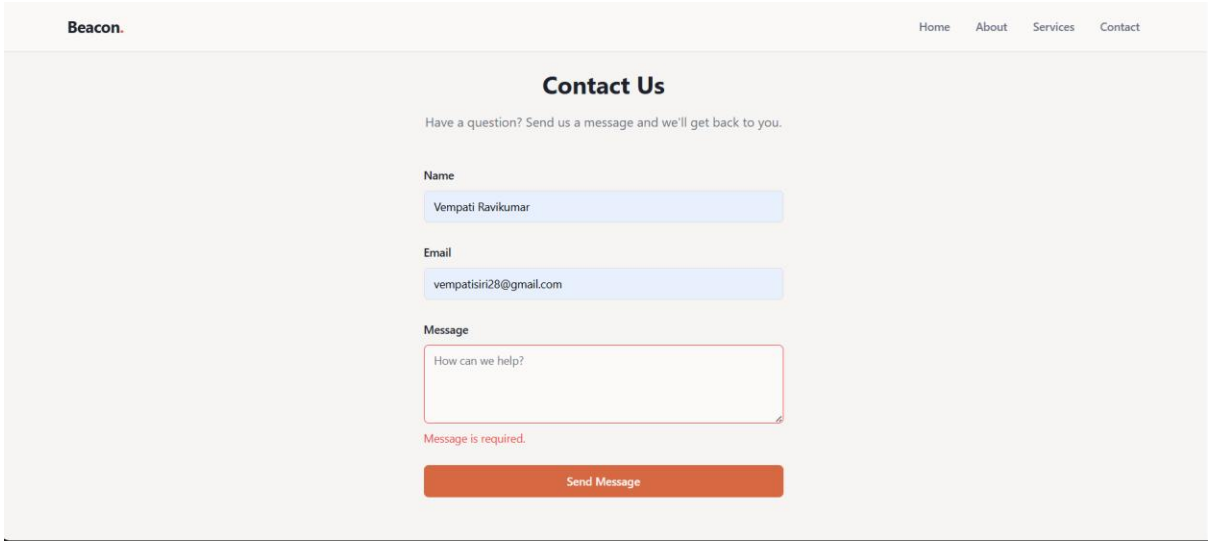
- Name
- Email
- Message

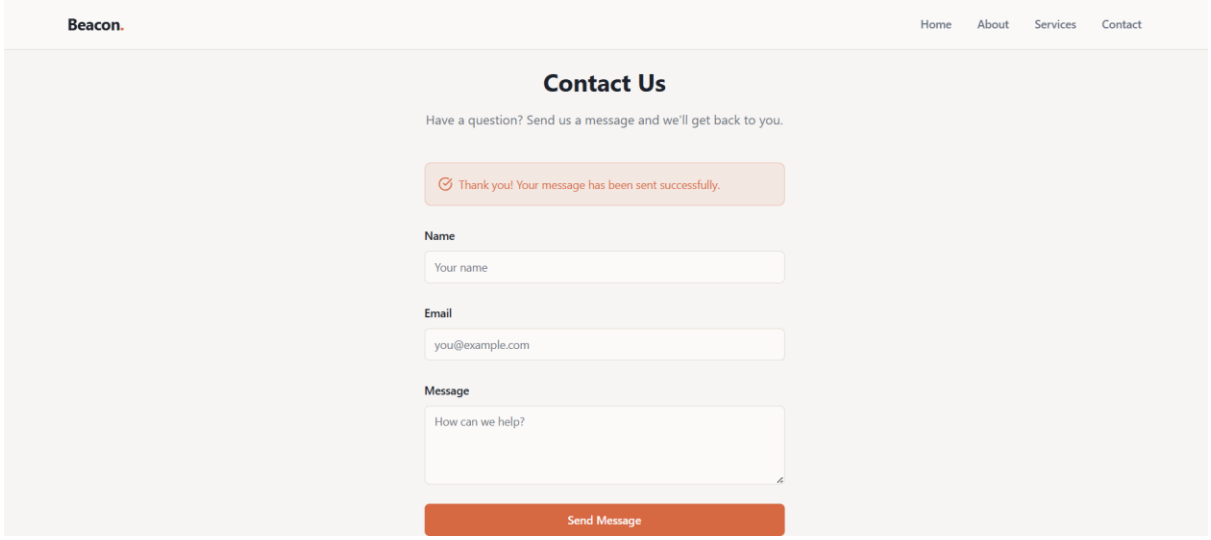
Add JavaScript client-side validation that:

- Prevents empty form submissions
- Validates the email format
- Displays clear inline error messages below each field if the input is invalid
- Shows a success message when all inputs are valid

Ensure the code is simple, well structured, and includes comments explaining the validation logic.

Output:





Explanation:

This project creates a contact form with client-side validation using HTML and JavaScript.

- The form includes Name, Email, and Message fields.
- JavaScript checks if any field is empty when the form is submitted.
- It also uses a regular expression to validate the email format.
- If the input is invalid, inline error messages appear below the fields.
- If all inputs are correct, a success message is displayed.
- The form uses event.preventDefault() so validation occurs without reloading the page.

Task 4: Dynamic List Management for a

Product Page

Scenario

A product listing page needs to add or remove items dynamically

without refreshing the page.

Task Description

• Create a simple HTML list to display items.

• Use AI to generate JavaScript that:

o Adds a new item when an “Add” button is clicked

o Removes the last item when a “Remove” button is clicked

• Ensure the DOM updates dynamically.

Expected Output

• A web page where:

o Items can be added and removed dynamically

o Changes appear instantly without page reload

o JavaScript logic is clear and maintainable

Prompt:

Create a simple product list web page using HTML and JavaScript.

Requirements:

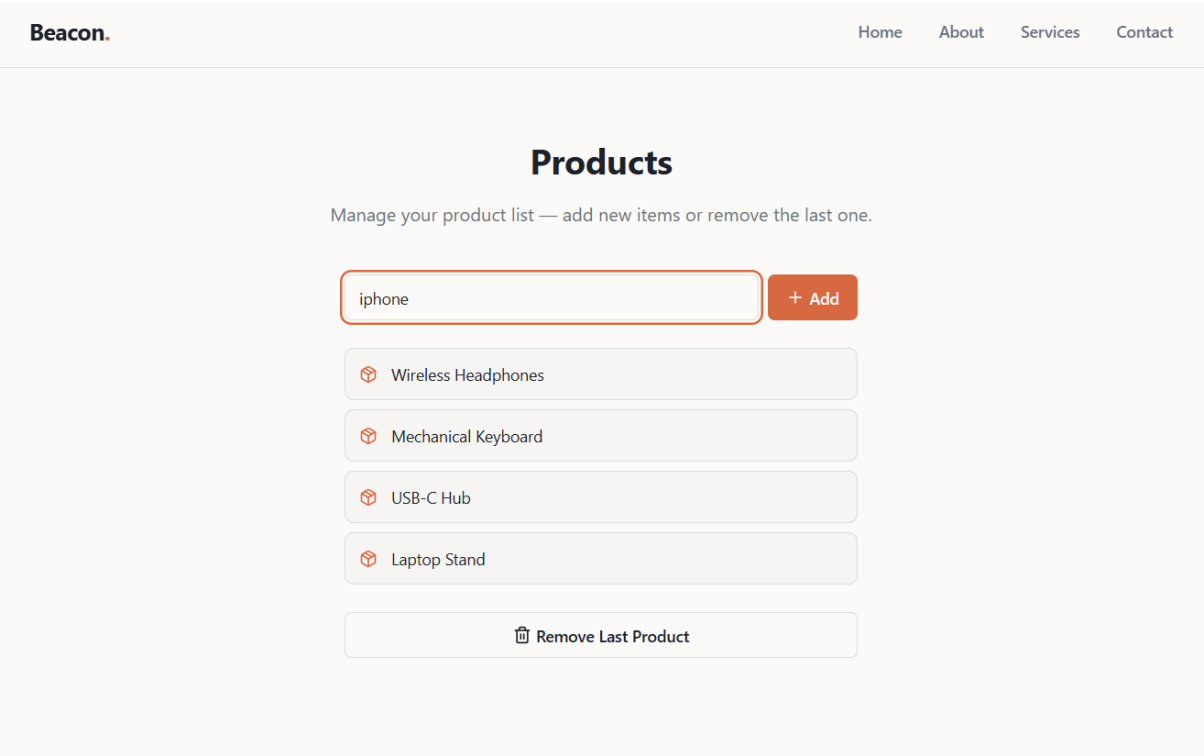
- Display a list of products using an HTML unordered list.
- Add an input field to enter a new product name.
- Include two buttons:
    1. Add Product
    2. Remove Last Product

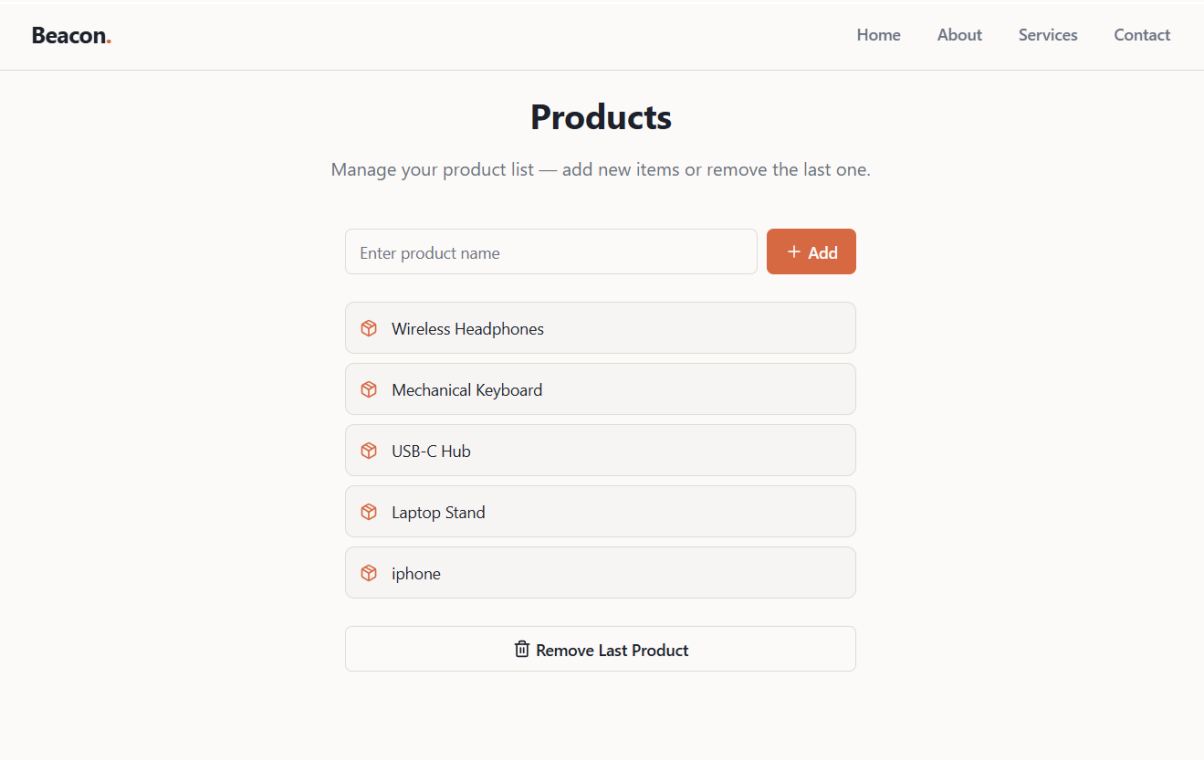
JavaScript should:

- Add a new list item when the Add button is clicked.
- Remove the last item when the Remove button is clicked.
- Update the DOM dynamically without reloading the page.

Ensure the code is simple, readable, and includes comments explaining the logic.

Output:





Explanation:

This project demonstrates dynamic list management using JavaScript DOM manipulation.

- An HTML unordered list (<ul>) displays product items.
- An input field allows users to type a new product name.
- The Add Product button creates a new <li> element and adds it to the list.

- The Remove Last Product button deletes the last item in the list.

JavaScript updates the DOM dynamically, so products are added or removed instantly without reloading the page.

The code uses functions like:

- createElement() to create new list items
- appendChild() to add items
- removeChild() to remove items

Task 5: Modal Popup for User Notifications

Scenario

You are building a user notification popup for announcements or alerts

on a website.

Task Description

• Use AI to help generate:

o HTML structure for a modal popup

o CSS styling with a semi-transparent background overlay

o JavaScript to open and close the modal

• Include multiple ways to close the modal (button or overlay

click).

Expected Output

• A web page where:

o Clicking “Open Modal” displays the popup

o Popup overlays the page content clearly

o Modal closes smoothly when instructed

Prompt:

Generate a modal popup notification using HTML, CSS, and JavaScript.

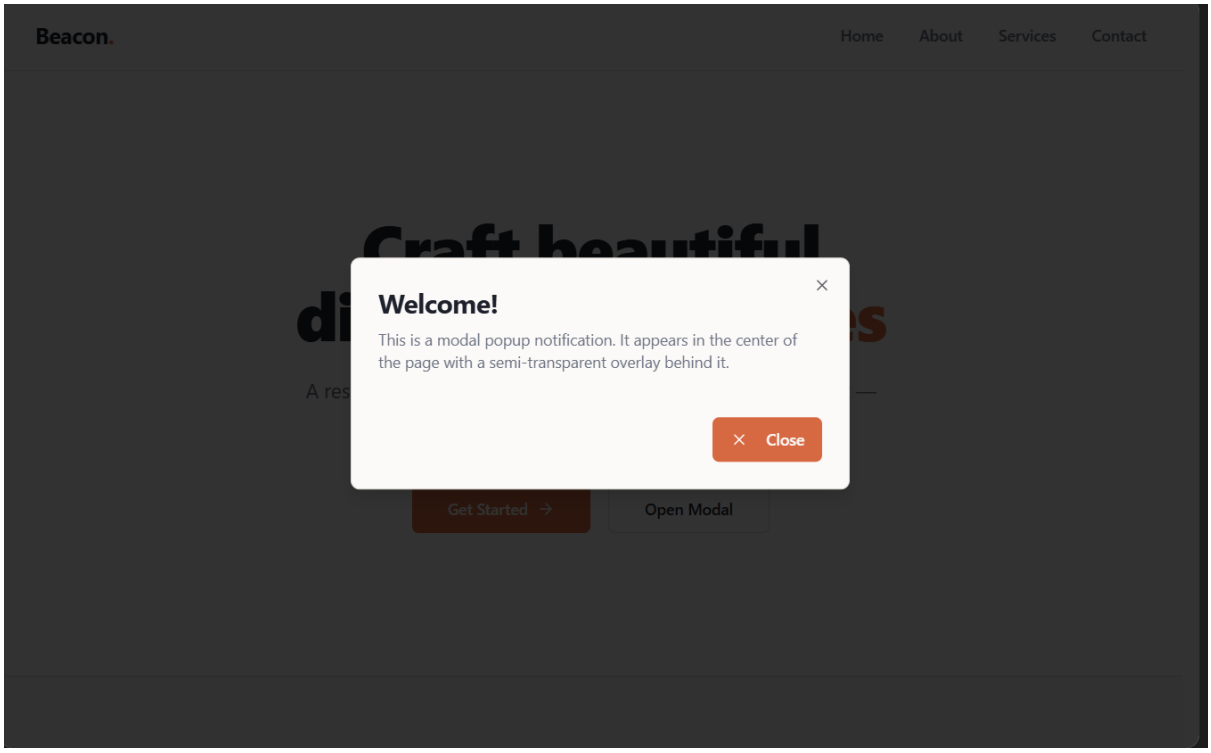
Requirements:

- Add a button labeled "Open Modal".
- When clicked, a popup modal should appear in the center of the page.
- The modal should have a semi-transparent background overlay.
- Include a close button inside the modal.

- The modal should also close when the user clicks outside the popup (overlay).

Ensure the code is simple, clean, and beginner-friendly.

Output:



Explanation:

This project creates a modal popup notification using HTML, CSS, and JavaScript.

- The HTML defines the modal structure and the Open Modal button.
- CSS styles the modal with a semi-transparent overlay and centers the popup on the page.
- JavaScript controls the modal behavior:
    - Clicking Open Modal displays the popup.
    - Clicking the Close button hides the modal.
    - Clicking outside the modal (overlay) also closes it.

The popup appears above the page content and closes smoothly without refreshing the page